SENATI

Escuela de Tecnologías de la Información

Laboratorio #4 · Guía Práctica 4

Creación de mi portafolio en React usando componentes



Curso	Fullstack Developer Software – Seminario (IA 501)
Instructor	Ing. Giancarlos Barboza N.
Alumno	Dorian Girón Prato
Carrera	Ingeniería de Software con Inteligencia Artificial · 5.º ciclo
Repositorio	github.com/Dgirto/dorian-portfolio
Fecha	19 de octubre de 2026

Documentación con capturas de código y de ejecución.

Contenido

Índice

1. Objetivo, tecnologías y estructura del proyecto
2. Pasos 1 a 4 · Navbar (cabecera con menú hamburguesa)
3. Pasos 5 a 7 · Banner (portada animada)
4. Pasos 8 a 10 · Perfil (sobre mí y formación)
5. Pasos 11 a 13 · Habilidades
6. Pasos 14 a 16 · Proyectos
7. Pasos 17 a 19 · Contacto (formulario controlado)
8. Ensamblaje final en App y vista móvil
9. Proyecto adicional: alerta sísmica con Arduino (Tinkercad)
10. Verificación: lint, tipos, pruebas, compilación y Lighthouse
11. Diferencias con la guía y conclusiones

1. Objetivo

Construir un portafolio personal en React dividido en componentes reutilizables, siguiendo la estructura de la guía: Navbar, Banner, Perfil, Habilidades, Proyectos y Contacto, cada uno con su lógica y sus estilos encapsulados y ensamblados en `App`.

El resultado es mi portafolio profesional real, publicado en GitHub: muestra mi experiencia como ingeniero de software júnior en Ruvic AI, mi formación en SENATI (5.º ciclo de Ingeniería de Software con IA) y mis proyectos con código público.

Tecnologías

Herramienta	Uso en el proyecto
React 19 + Vite	Componentes, estado con `useState`, efectos con `useEffect` y servidor de desarrollo.
TypeScript	Tipos para los datos (proyectos, habilidades, trayectoria) y los componentes.
CSS Modules	Un archivo `.module.css` por componente: estilos encapsulados, como pide la guía.
Motion	Animaciones al hacer scroll (respetando «reducir movimiento»).
three.js	Portátil 3D del banner, que se abre con el scroll.
Vitest + ESLint	Pruebas automáticas de los datos y análisis estático del código.

Paso 1 y 2

Instalación y estructura de archivos

El proyecto se creó con Vite (plantilla React + TypeScript). Igual que en la guía, cada componente tiene su lógica (`.tsx`) y sus estilos (`.module.css`) en archivos separados. Los datos (proyectos, habilidades, trayectoria y perfil) están centralizados en `src/data` , así que añadir un proyecto no exige tocar los componentes.

Explorador · src/

```
src/
├── components/
│   ├── CodeWindow.module.css
│   ├── CodeWindow.tsx
│   ├── ContactForm.module.css
│   ├── ContactForm.tsx
│   ├── Counter.tsx
│   ├── Devices.module.css
│   ├── Devices.tsx
│   ├── ExternalLink.tsx
│   ├── LaptopCanvas.module.css
│   ├── LaptopCanvas.tsx
│   ├── LogoTile.module.css
│   ├── LogoTile.tsx
│   ├── Reveal.tsx
│   ├── ScrollWords.module.css
│   ├── ScrollWords.tsx
│   ├── SiteHeader.module.css
│   └── SiteHeader.tsx
├── data/
│   ├── certifications.ts
│   ├── connectors.ts
│   ├── data.test.ts
│   ├── families.ts
│   ├── navigation.ts
│   ├── profile.ts
│   ├── projects.ts
│   ├── stack.ts
│   └── timeline.ts
├── hooks/
│   ├── useGithubSummary.ts
│   └── useLocalTime.ts
├── sections/
│   ├── About.module.css
│   ├── About.tsx
│   ├── ConnectorWall.module.css
│   ├── ConnectorWall.tsx
│   ├── Contact.module.css
│   ├── Contact.tsx
│   ├── Explorer.module.css
│   ├── Explorer.tsx
│   ├── Footer.module.css
│   ├── Footer.tsx
│   ├── GitHubLive.module.css
│   ├── GitHubLive.tsx
│   ├── Hero.module.css
│   ├── Hero.tsx
│   ├── Journey.module.css
│   ├── Journey.tsx
│   ├── Projects.module.css
│   ├── Projects.tsx
│   ├── Stack.module.css
│   ├── Stack.tsx
│   └── Statement.tsx
├── styles/
│   └── global.css
├── three/
│   └── laptopScene.ts
├── types/
│   └── index.ts
├── utils/
│   ├── connectors.ts
│   ├── format.test.ts
│   ├── format.ts
│   ├── github.test.ts
│   ├── github.ts
│   ├── search.test.ts
│   └── search.ts
├── App.tsx
└── main.tsx
```

package.json · líneas 1–13

```
1  {
2    "name": "dorian-portfolio",
3    "private": true,
4    "version": "1.0.0",
5    "type": "module",
6    "description": "Personal portfolio built with React and TypeScript",
7    "scripts": {
8      "dev": "vite",
9      "build": "tsc -b && vite build",
10     "preview": "vite preview",
11     "lint": "eslint .",
12     "typecheck": "tsc -b --noEmit",
13     "test": "vitest run"
```

Terminal · dorian-portfolio

```
> node --version && npm --version
v24.15.0
12.0.2
```

Pasos 1 a 4

Navbar: cabecera con menú hamburguesa

Equivale a `Navbar.jsx` de la guía. Con `useState` se controla si el menú móvil está abierto (`open`), y al pulsar un enlace se cierra con `setOpen(false)` . Además, un `IntersectionObserver` resalta la sección visible y la tecla Escape cierra el menú. Los enlaces salen de un arreglo de datos recorrido con `map` .

src/components/SiteHeader.tsx

```tsx
import { useEffect, useId, useState } from 'react'
import { Menu, X } from 'lucide-react'
import { profile } from '../data/profile'
import { navItems } from '../data/navigation'
import styles from './SiteHeader.module.css'

/** Barra fija y translúcida con la sección activa resaltada. */
export function SiteHeader() {
  const [open, setOpen] = useState(false)
  const [active, setActive] = useState<string>('')
  const menuId = useId()

  useEffect(() => {
    if (!open) return
    const onKey = (e: KeyboardEvent) => e.key === 'Escape' && setOpen(false)
    window.addEventListener('keydown', onKey)
    return () => window.removeEventListener('keydown', onKey)
  }, [open])

  useEffect(() => {
    const sections = navItems
      .map((n) => document.querySelector(n.href))
      .filter((el): el is Element => el !== null)
    const observer = new IntersectionObserver(
      (entries) => {
        for (const e of entries) if (e.isIntersecting) setActive(`#${e.target.id}`)
      },
      { rootMargin: '-45% 0px -50% 0px' },
    )
    sections.forEach((s) => observer.observe(s))
    return () => observer.disconnect()
  }, [])

  return (
    <header className={styles.header}>
      <div className={styles.bar}>
        <a className={styles.wordmark} href="#inicio" onClick={() => setOpen(false)}>
          {profile.shortName}
        </a>
        <nav aria-label="Principal" className={styles.nav}>
          <ul id={menuId} className={`${styles.links} ${open ? styles.open : ''}`}>
            {navItems.map((item) => (
              <li key={item.href}>
                <a
                  href={item.href}
                  aria-current={active === item.href ? 'true' : undefined}
                  onClick={() => setOpen(false)}
                >
                  {item.label}
                </a>
              </li>
            ))}
          </ul>
```

```
          <a className={styles.action} href={`mailto:${profile.email}`}>
            Escríbeme
          </a>
          <button
            type="button"
            className={styles.toggle}
            aria-expanded={open}
            aria-controls={menuId}
            aria-label={open ? 'Cerrar menú' : 'Abrir menú'}
            onClick={() => setOpen((v) => !v)}
          >
            {open ? <X size={20} aria-hidden="true" /> : <Menu size={20} aria-hidden="true" />}
          </button>
        </nav>
      </div>
    </header>
  )
}
```

src/data/navigation.ts · líneas 1–9

```
// Enlaces de la navegación principal (cabecera) y del pie.
export const navItems = [
  { href: '#sobre-mi', label: 'Sobre mí' },
  { href: '#proyectos', label: 'Proyectos' },
  { href: '#conectores', label: 'Conectores' },
  { href: '#stack', label: 'Stack' },
  { href: '#trayectoria', label: 'Trayectoria' },
  { href: '#contacto', label: 'Contacto' },
]
```

Estilos y responsividad (Paso 3)

En pantallas de hasta 900 px los enlaces se ocultan y aparece el botón hamburguesa; con la clase `open` el menú se despliega.

src/components/SiteHeader.module.css · líneas 93–134

```
@media (max-width: 900px) {
  .toggle {
    display: inline-flex;
  }

  .nav {
    gap: 8px;
  }

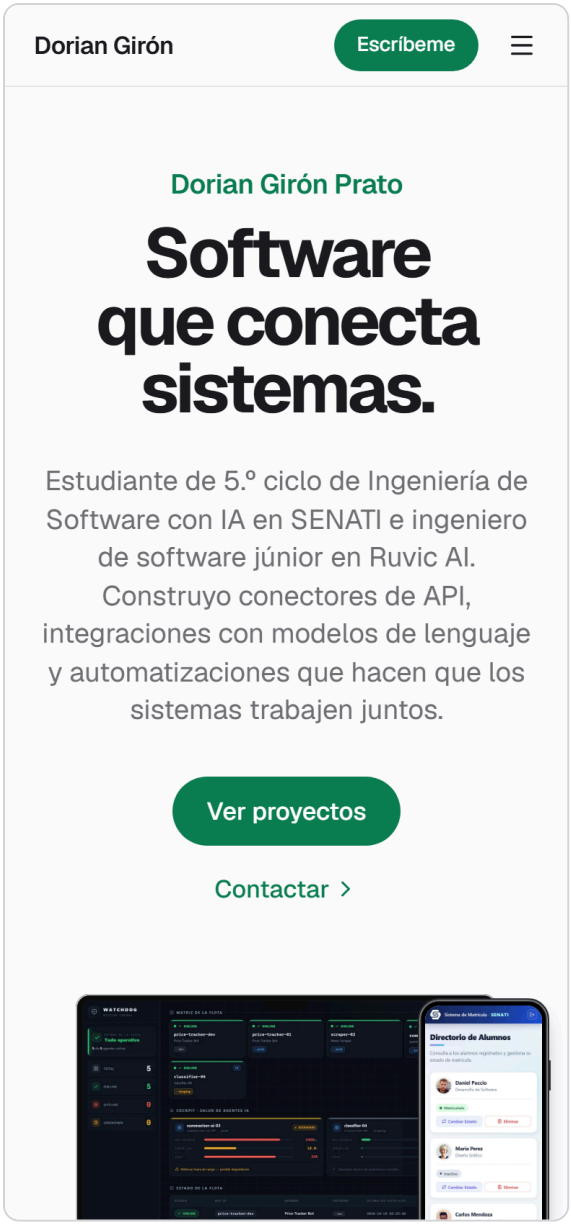
  .links {
    display: none;
    position: absolute;
    top: 56px;
    left: 0;
    right: 0;
    flex-direction: column;
    gap: 0;
    padding: 8px var(--margin) 28px;
    background: var(--bg);
    font-size: 1.5rem;
    font-weight: 600;
    letter-spacing: -0.02em;
    box-shadow: 0 24px 40px -24px rgb(0 0 0 / 0.25);
  }
```

```
117
118   .links.open {
119     display: flex;
120   }
121
122   .links a {
123     min-height: 52px;
124     width: 100%;
125     padding: 0;
126     color: var(--ink);
127     border-radius: 0;
128   }
129
130   .links a[aria-current='true'] {
131     background: none;
132     color: var(--accent-strong);
133   }
134 }
```

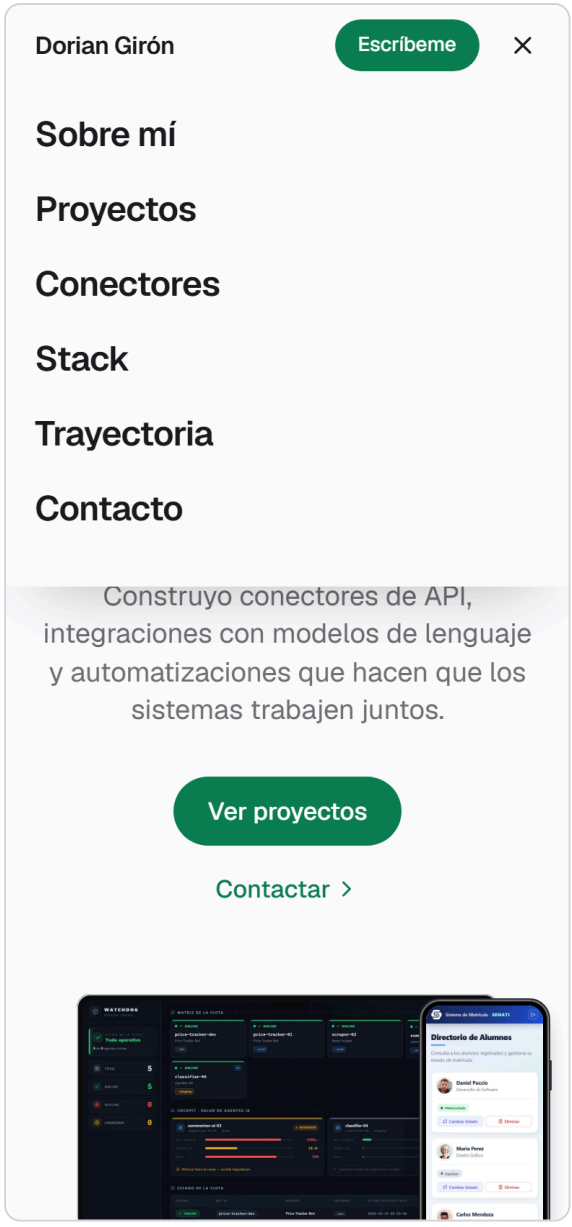
Ejecución



Cabecera fija y translúcida en escritorio, con el enlace «Escríbeme».



Móvil: menú cerrado.



Móvil: menú hamburguesa abierto (estado `open = true`).

Pasos 5 a 7

Banner: portada con portátil que se abre al hacer scroll

Equivale a `Banner.jsx`. Muestra mi nombre, una frase y los botones de acción. En lugar de la foto circular de la guía, el banner presenta un portátil 3D (three.js) que llega cerrado y se abre según el progreso del scroll (`useScroll` + `useTransform`), y al final aparece un teléfono con otro de mis proyectos. Si el usuario prefiere reducir el movimiento, se muestra una imagen fija.

src/sections/Hero.tsx · líneas 11–54

```tsx
export function Hero() {
  const trackRef = useRef<HTMLDivElement>(null)
  const reduce = useReducedMotion()
  // Recorrido: el portátil llega cerrado y se abre mientras la escena queda fijada.
  const { scrollYProgress } = useScroll({ target: trackRef, offset: ['start start', 'end end'] })
  const smooth = useSpring(scrollYProgress, { stiffness: 140, damping: 30, mass: 0.4 })
  const open = useTransform(smooth, [0, 0.62], [0, 1], { clamp: true })
  const phoneY = useTransform(smooth, [0.55, 0.85], ['40%', '0%'], { clamp: true })
  const phoneOpacity = useTransform(smooth, [0.55, 0.75], [0, 1], { clamp: true })
  const hint = useTransform(smooth, [0, 0.15], [1, 0], { clamp: true })

  const enter = (delay: number) =>
    reduce
      ? {}
      : {
          // Solo desplazamiento: el texto es visible desde el primer fotograma (mejor LCP).
          initial: { y: 24 },
          animate: { y: 0 },
          transition: { duration: 1, delay, ease },
        }

  return (
    <section id="inicio" className={styles.hero} aria-labelledby="hero-title">
      <div className={styles.copy}>
        <h1 id="hero-title" className={styles.title}>
          <m.span className={styles.name} {...enter(0)}>
            {profile.name}
          </m.span>
          <m.span className={styles.claim} {...enter(0.1)}>
            Software que conecta sistemas.
          </m.span>
        </h1>
        <m.p className={styles.intro} {...enter(0.25)}>
          {profile.intro}
        </m.p>
        <m.div className={styles.actions} {...enter(0.35)}>
          <a className="btn" href="#proyectos">
            Ver proyectos
          </a>
          <a className="link-more" href="#contacto">
            Contactar <ChevronRight size={18} aria-hidden="true" />
          </a>
        </m.div>
      </div>
```

Estilos (Paso 6)

src/sections/Hero.module.css · líneas 52–79

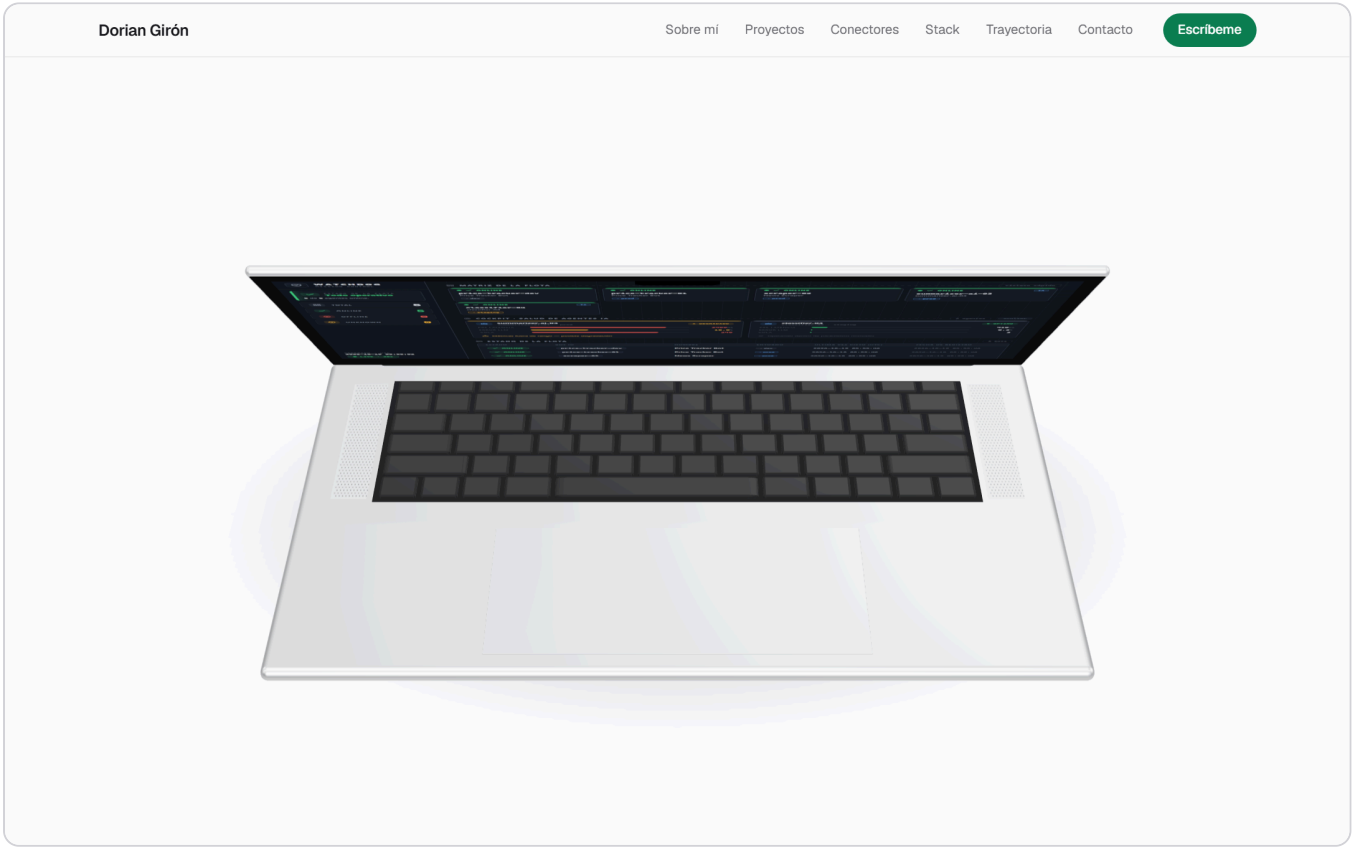
```css
.track {
  height: 230vh;
  margin-top: clamp(24px, 4vh, 48px);
}

.trackStatic {
  margin-top: clamp(40px, 6vh, 72px);
  padding-bottom: clamp(48px, 6vw, 96px);
}

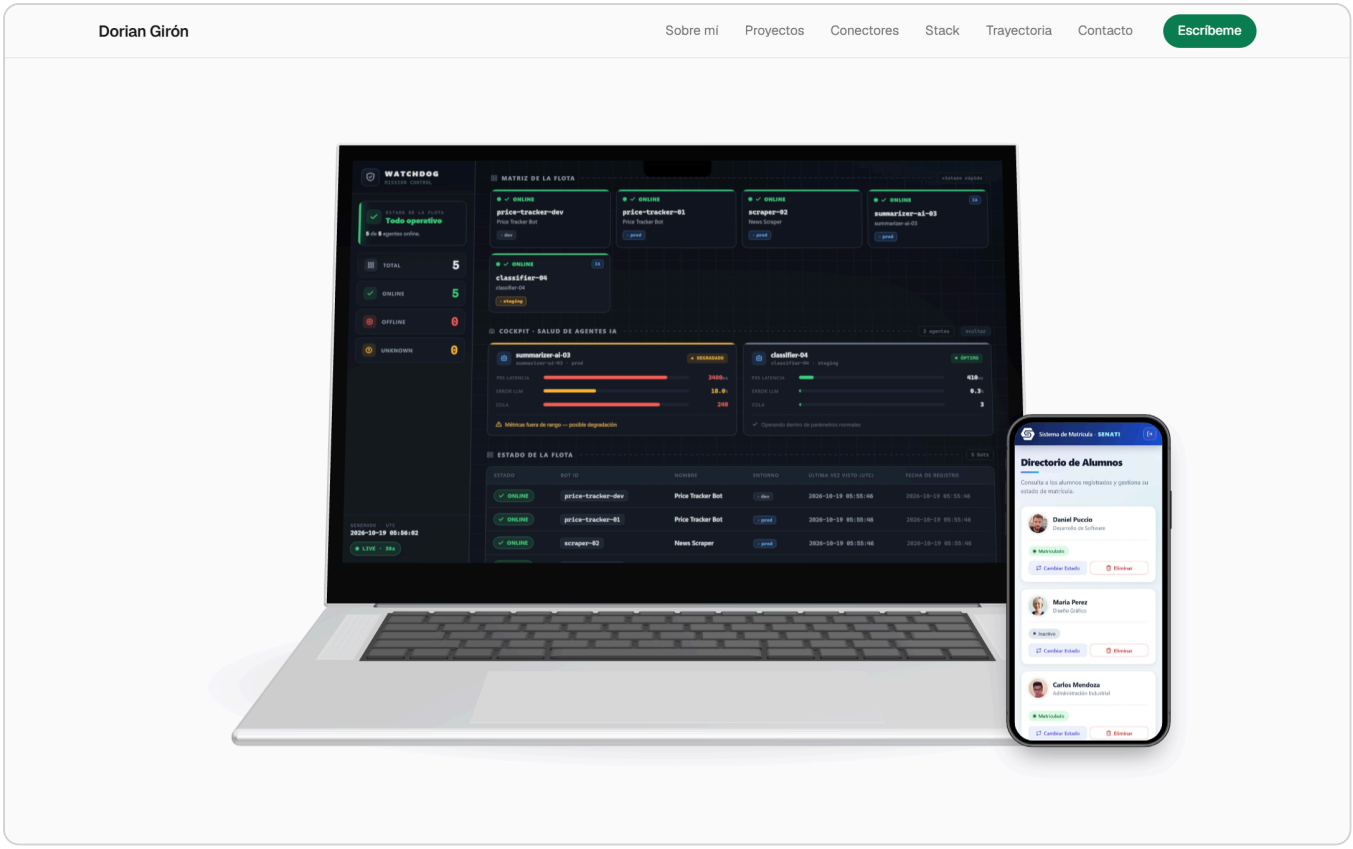
.sticky {
  position: sticky;
  top: 56px;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  height: calc(100vh - 56px);
  height: calc(100svh - 56px);
  overflow: hidden;
}

.trackStatic .sticky {
  position: static;
  height: auto;
  overflow: visible;
}
```

Ejecución



El portátil abriéndose mientras se hace scroll.



Portátil abierto con el panel de watchdog-bot-monitor y el teléfono con appEscuela.

Pasos 8 a 10

Perfil: sobre mí y formación

Equivale a `Perfil.jsx`. Los textos salen de `src/data/profile.ts`; la foto tiene un leve efecto de paralaje. La formación académica y la experiencia se muestran en la sección Trayectoria a partir de `src/data/timeline.ts`.

src/data/profile.ts · líneas 1–20

```ts
import type { Profile } from '../types'

// Datos verificados en LinkedIn (perfil propio, 2026-10-19) y GitHub (Dgirto).
// Edita aquí cualquier dato personal; ningún componente repite estos textos.
export const profile: Profile = {
  name: 'Dorian Girón Prato',
  shortName: 'Dorian Girón',
  headline: 'Software Developer Intern en Ruvic AI | Ingeniería de Software con Inteligencia Artificial',
  role: 'Ingeniero de software júnior en Ruvic AI',
  studies: 'Estudiante de 5.º ciclo de Ingeniería de Software con IA en SENATI',
  location: 'Lima, Perú',
  workMode: 'Trabajo en remoto',
  intro:
    'Estudiante de 5.º ciclo de Ingeniería de Software con IA en SENATI e ingeniero de software júnior en Ruvic AI. Construyo conectores de API, integraciones con modelos de lenguaje y automatizaciones que hacen que los sistemas trabajen juntos.',
  about:
    'Curso el 5.º ciclo de Ingeniería de Software con Inteligencia Artificial en SENATI y, desde marzo de 2026, hago mis prácticas como ingeniero de software júnior en Ruvic AI. Mi trabajo va del frontend web a los servicios del backend: construyo en Python los conectores que permiten a un agente de IA consultar bases de datos, leer correo u operar servicios en la nube, y los pruebo antes de cada entrega.',
  aboutMore:
    'Me interesa entender cómo funciona el código por dentro y convertir procesos manuales en flujos automáticos. Fuera del trabajo sigo construyendo: aplicaciones web y móviles, herramientas de monitoreo y prototipos IoT simulados en Tinkercad.',
  statement:
    'Hoy un agente de IA puede leer tu correo, consultar tu base de datos o revisar tu firewall. Antes necesita una forma segura de conectarse. Ese es mi trabajo.',
```

src/sections/About.tsx

```tsx
import { useRef } from 'react'
import { m, useReducedMotion, useScroll, useTransform } from 'motion/react'
import { profile } from '../data/profile'
import { Reveal } from '../components/Reveal'
import styles from './About.module.css'

const facts = [
  { label: 'Base', value: profile.location },
  { label: 'Ahora', value: 'Prácticas en Ruvic AI' },
  { label: 'Formación', value: 'SENATI · 5.º ciclo' },
]

export function About() {
  const ref = useRef<HTMLElement>(null)
  const reduce = useReducedMotion()
  const { scrollYProgress } = useScroll({ target: ref, offset: ['start end', 'end start'] })
  const photoY = useTransform(scrollYProgress, [0, 1], [30, -30])

  return (
    <section ref={ref} id="sobre-mi" className={styles.about} aria-labelledby="sobre-mi-title">
      <div className={styles.grid}>
```

```
        <Reveal className={styles.photoWrap}>
          <m.img
            className={styles.photo}
            src={profile.portrait.src}
            alt={profile.portrait.alt}
            width={profile.portrait.width}
            height={profile.portrait.height}
            loading="lazy"
            decoding="async"
            style={reduce ? undefined : { y: photoY }}
          />
        </Reveal>
        <div className={styles.text}>
          <Reveal>
            <h2 id="sobre-mi-title" className="section-title">
              Sobre mí
            </h2>
          </Reveal>
          <Reveal delay={0.1}>
            <p className={styles.lead}>{profile.about}</p>
          </Reveal>
          <Reveal delay={0.15}>
            <p className={styles.more}>{profile.aboutMore}</p>
          </Reveal>
          <Reveal delay={0.2}>
            <dl className={styles.facts}>
              {facts.map((f) => (
                <div key={f.label}>
                  <dt>{f.label}</dt>
                  <dd>{f.value}</dd>
                </div>
              ))}
            </dl>
          </Reveal>
        </div>
      </div>
    </section>
  )
}
```

src/data/timeline.ts

```
import type { TimelineEntry } from '../types'

// Fuente: LinkedIn (verificado el 2026-10-19). Solo datos confirmados.
export const timeline: TimelineEntry[] = [
  {
    kind: 'experiencia',
    title: 'Ingeniero de software júnior',
    organization: 'Ruvic AI',
    type: 'Contrato de prácticas',
    start: '2026-03',
    end: null,
    location: 'En remoto',
    description:
      'Desarrollo de software y aseguramiento de calidad: interfaces web con HTML, CSS, JavaScript y TypeScript; servicios core y conectores internos en Python; pruebas QA antes de cada despliegue y control de versiones en GitHub.',
  },
  {
    kind: 'formacion',
    title: 'Profesional Técnico, Ingeniería de Software con Inteligencia Artificial',
    organization: 'SENATI',
    type: 'Formación académica · cursando el 5.º ciclo',
```

```
    start: '2024-09',
    end: '2027-07',
    endIsExpected: true,
  },
]
```

Estilos (Paso 9)

src/sections/About.module.css · líneas 1–33

```
.about {
  padding-block: var(--section);
}

.grid {
  display: grid;
  grid-template-columns: minmax(0, 5fr) minmax(0, 7fr);
  gap: clamp(32px, 6vw, 96px);
  align-items: center;
  width: var(--page);
  margin-inline: auto;
}

.photoWrap {
  overflow: hidden;
  border-radius: var(--radius);
  background: var(--surface-2);
  aspect-ratio: 3 / 4;
  max-width: 420px;
}

.photo {
  width: 100%;
  height: 112%;
  margin-top: -6%;
  object-fit: cover;
}

.text {
  display: grid;
  gap: 24px;
}
```

Ejecución



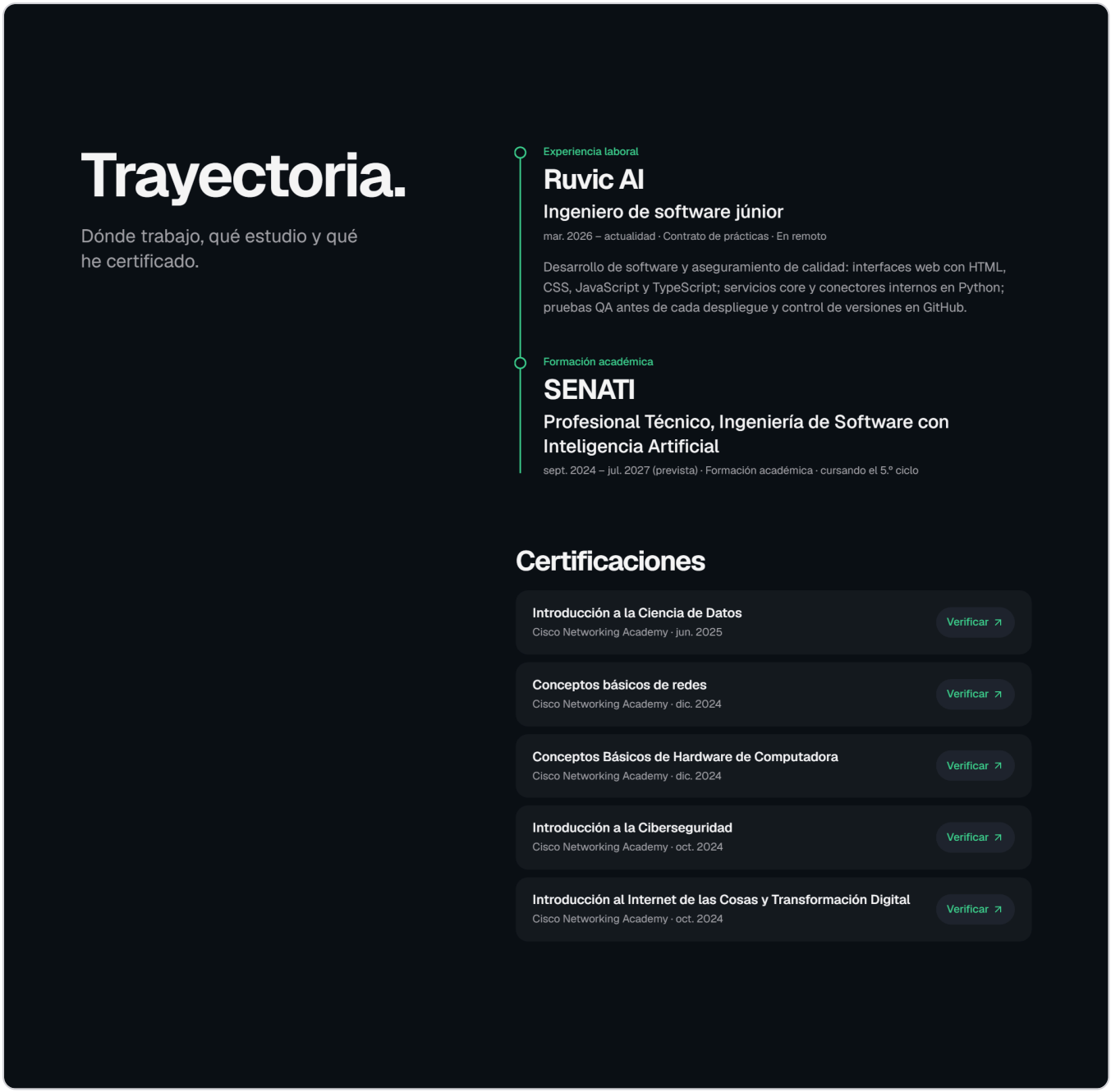
Sobre mí

Curso el 5.º ciclo de Ingeniería de Software con Inteligencia Artificial en SENATI y, desde marzo de 2026, hago mis prácticas como ingeniero de software júnior en Ruvic AI. Mi trabajo va del frontend web a los servicios del backend: construyo en Python los conectores que permiten a un agente de IA consultar bases de datos, leer correo u operar servicios en la nube, y los pruebo antes de cada entrega.

Me interesa entender cómo funciona el código por dentro y convertir procesos manuales en flujos automáticos. Fuera del trabajo sigo construyendo: aplicaciones web y móviles, herramientas de monitoreo y prototipos IoT simulados en Tinkercad.

Base	Ahora	Formación
Lima, Perú	**Prácticas en Ruvic AI**	**SENATI · 5.º ciclo**

Sección «Sobre mí»: 5.º ciclo de Ingeniería de Software con IA en SENATI y prácticas en Ruvic AI.



Trayectoria: experiencia, formación en SENATI y certificaciones verificables.

Pasos 11 a 13

Habilidades

Equivale a `Habilidades.jsx`: un arreglo de datos recorrido con `map`. En vez de porcentajes, cada tecnología indica dónde se puede comprobar (repositorio o proyecto), para no inventar niveles.

src/data/stack.ts · líneas 1–28

```ts
import type { StackGroup } from '../types'

const gh = (repo: string) => `https://github.com/Dgirto/${repo}`

// Sin niveles ni porcentajes: cada tecnología dice dónde se ve.
// "Sin repositorio público aún" marca conocimientos confirmados que todavía no tienen código público.
export const stack: StackGroup[] = [
  {
    name: 'Lenguajes',
    items: [
      { name: 'Python', evidence: '46 conectores, watchdog', href: gh('watchdog-bot-monitor') },
      { name: 'TypeScript', evidence: 'Ruvic AI, este sitio' },
      { name: 'JavaScript', evidence: 'appEscuela', href: gh('appEscuela') },
      { name: 'Java', evidence: 'appTareas (Android)', href: gh('appTareas') },
      { name: 'C y C++', evidence: 'Sin repositorio público aún' },
      { name: 'SQL', evidence: 'Conectores de bases de datos', href: gh('PostgreSQL') },
    ],
  },
  {
    name: 'Desarrollo frontend',
    items: [
      { name: 'React', evidence: 'appEscuela, este sitio', href: 'https://app-escuela-rho.vercel.app' },
      { name: 'HTML y CSS', evidence: 'Guías de los conectores', href: gh('Gmail') },
      { name: 'Vite', evidence: 'appEscuela, este sitio', href: gh('appEscuela') },
      { name: 'Bootstrap 5', evidence: 'appEscuela', href: gh('appEscuela') },
      { name: 'Android (XML, Material)', evidence: 'appTareas', href: gh('appTareas') },
    ],
  },
```

src/sections/Stack.tsx · líneas 25–58

```tsx
export function Stack() {
  return (
    <section id="stack" className={`${styles.section} defer`} aria-labelledby="stack-title">
      <div className={styles.head}>
        <Reveal>
          <h2 id="stack-title" className="section-title">
            Stack, con evidencia.
          </h2>
        </Reveal>
        <Reveal delay={0.1}>
          <p className="section-lead">
            Sin porcentajes inventados: cada herramienta indica dónde puedes verla en uso.
          </p>
        </Reveal>
      </div>

      <ul className={styles.bento}>
        {stack.map((group, i) => (
          <Reveal as="li" key={group.name} delay={(i % 3) * 0.08} className={`${styles.tile}
${styles[spans[i] ?? 'normal']}`}>
            <h3 className={styles.tileTitle}>{group.name}</h3>
            <ul className={styles.items}>
```

```
              {group.items.map((item) => (
                <li key={item.name}>
                  <span className={styles.name}>{item.name}</span>
                  <Evidence item={item} />
                </li>
              ))}
            </ul>
          </Reveal>
        ))}
      </ul>
    </section>
  )
}
```

Estilos y cuadrícula (Paso 12)

src/sections/Stack.module.css · líneas 10–32

```
.bento {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 20px;
  width: var(--page);
  margin: clamp(40px, 6vw, 72px) auto 0;
}

.tile {
  padding: clamp(24px, 2.6vw, 36px);
  border-radius: var(--radius);
  background: var(--surface);
}

.wide {
  grid-column: span 2;
}

.wide .items {
  columns: 2;
  column-gap: 40px;
}
```

Ejecución

Stack, con evidencia.

Sin porcentajes inventados: cada herramienta indica dónde puedes verla en uso.

Lenguajes

Python
46 conectores, watchdog

TypeScript
Ruvic AI, este sitio

JavaScript
appEscuela

Java
appTareas (Android)

C y C++
Sin repositorio público aún

SQL
Conectores de bases de datos

Desarrollo frontend

React
appEscuela, este sitio

HTML y CSS
Guías de los conectores

Vite
appEscuela, este sitio

Bootstrap 5
appEscuela

Android (XML, Material)
appTareas

Inteligencia artificial y APIs

Integración de LLM con API keys
Sin repositorio público aún

Herramientas para agentes de IA
SKILL.md de cada conector

Métricas de agentes de IA
watchdog: latencia p95, error LLM

Microsoft Graph
msgraph-Robin

Google Workspace APIs
Gmail, Drive, Sheets, Docs, Calendar

AWS (boto3) y Azure SDK
Conectores de nube

OAuth2
Gmail, Drive OAuth2

FastAPI y WebSockets
watchdog-bot-monitor

Visión por computadora (OpenCV, EasyOCR)
parqueo

Automatización

Automatización de procesos
Sin repositorio público aún

Alertas por correo y Slack
watchdog: SendGrid, webhooks

Lectura periódica de correo
msgraph-Robin: polling

Integración de servicios por API
46 conectores

Infraestructura y contenedores

Docker
Pruebas locales de conectores

LocalStack
Conectores AWS

PostgreSQL y SQLite
watchdog, appTareas

Permisos mínimos (IAM, roles)
README de cada conector

Vercel
appEscuela

IoT y sistemas embebidos

Simulación de circuitos en Tinkercad
Sin repositorio público aún

Soluciones IoT
Certificación Cisco IoT

Redes y hardware
Certificaciones Cisco

Herramientas de desarrollo

Git y GitHub
github.com/Dgirto

pytest
watchdog: 29 pruebas

Gradle y Android Studio
appTareas

Figma
Diseño de appTareas

Pruebas QA
Ruvic AI

Stack con evidencia: cada habilidad enlaza al lugar donde se usa.

Pasos 14 a 16

Proyectos

Equivale a `Proyectos.jsx`. Cada proyecto es un objeto con imagen, descripción, etiquetas de tecnologías y enlaces al código y a la demo. La cuadrícula se genera con `map`; al pulsar una tarjeta se abre un diálogo con galería, el problema, la solución y mi aporte.

src/types/index.ts · tipo Project · líneas 50–69

```ts
export interface Project {
  slug: string
  name: string
  cover: ProjectCover
  /** Ocupa el ancho completo de la cuadrícula. */
  featured?: boolean
  kind: ProjectKind
  year: string
  /** Una frase-tesis: qué es, en una línea. */
  thesis: string
  problem: string
  solution: string
  contribution: string
  team: string
  stack: string[]
  slides: Slide[]
  repo: string
  demo?: string
  status?: string
}
```

src/data/projects.ts · líneas 1–30

```ts
import type { Project } from '../types'

// Proyectos destacados. Todo dato procede del repositorio público enlazado.
// Para añadir uno: copia un objeto, guarda sus capturas en public/work/ y enlázalas en `cover` y `slides`.
export const projects: Project[] = [
  {
    slug: 'watchdog',
    name: 'watchdog-bot-monitor',
    featured: true,
    kind: 'Personal',
    year: '2026',
    thesis: 'Monitoreo en tiempo real para bots y agentes de IA en producción.',
    problem: 'Un bot puede caerse sin que nadie lo note, y un agente de IA puede seguir en línea mientras responde mal.',
    solution:
      'Cada agente envía latidos firmados con HMAC por HTTP o WebSocket. Si deja de hacerlo, se abre un incidente y llega una alerta por correo o Slack, con filtros contra falsas alarmas. Un panel muestra el estado de la flota y la salud de cada agente de IA: latencia p95, tasa de error del LLM y cola.',
    contribution: 'Diseño y desarrollo completos: arquitectura hexagonal, API REST y WebSocket, panel web y 29 pruebas automatizadas.',
    team: 'Individual',
    stack: ['Python', 'FastAPI', 'WebSockets', 'SQLite / PostgreSQL', 'pytest'],
    repo: 'https://github.com/Dgirto/watchdog-bot-monitor',
    cover: {
      kind: 'laptop',
      src: '/work/watchdog-dashboard.webp',
      alt: 'Panel de watchdog-bot-monitor con cinco agentes en línea y la salud de dos agentes de IA',
      width: 1400,
      height: 875,
```

```
    },
    slides: [
      {
        type: 'image',
        src: '/work/watchdog-dashboard.webp',
```

src/sections/Projects.tsx · líneas 186–230

```tsx
  return (
    <section id="proyectos" className={`${styles.section} defer`} aria-labelledby="proyectos-title">
      <div className={styles.head}>
        <Reveal>
          <h2 id="proyectos-title" className="section-title">
            Proyectos.
          </h2>
        </Reveal>
        <Reveal delay={0.1}>
          <p className="section-lead">
            Trabajo real, con código público. Abre cualquiera para ver el problema, la solución y mi aporte.
          </p>
        </Reveal>
      </div>

      <ul className={styles.grid}>
        {projects.map((p, i) => (
          <Reveal as="li" key={p.slug} delay={(i % 2) * 0.08} className={p.featured ? styles.featured : undefined}>
            <article className={`${styles.card} ${p.featured ? styles.cardWide : ''}`}>
              <div className={styles.cardText}>
                <h3 className={styles.cardTitle}>
                  <button type="button" className={styles.cardButton} onClick={() => setOpen(p)} aria-haspopup="dialog">
                    {p.name}
                  </button>
                </h3>
                <p className={styles.cardThesis}>{p.thesis}</p>
                <p className={styles.kind}>
                  {p.kind} · {p.year}
                </p>
                <span className={styles.more} aria-hidden="true">
                  Ver proyecto <Plus size={16} />
                </span>
              </div>
              <div className={`${styles.cardMedia} ${styles[`media_${p.cover.kind}`] ?? ''}`}>
                <Cover project={p} />
              </div>
            </article>
          </Reveal>
        ))}
      </ul>

      <ProjectDialog project={open} onClose={() => setOpen(null)} />
    </section>
  )
}
```

Estilos (Paso 15)

src/sections/Projects.module.css · líneas 1–25

```css
.section {
  padding-block: var(--section);
}
```

```css

.head {
  width: var(--page);
  margin-inline: auto;
}

.grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 20px;
  width: var(--page);
  margin: clamp(40px, 6vw, 72px) auto 0;
}

.featured {
  grid-column: 1 / -1;
}

.grid > li {
  display: flex;
  min-width: 0;
}
```

Ejecución

Proyectos.

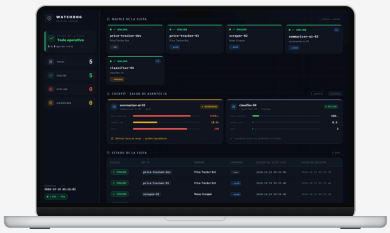
Trabajo real, con código público. Abre cualquiera para ver el problema, la solución y mi aporte.

watchdog-bot-monitor

Monitoreo en tiempo real para bots y agentes de IA en producción.

Personal · 2026

Ver proyecto +

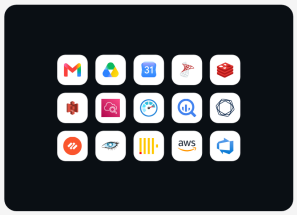


Catálogo de conectores de Ruvic AI

46 conectores que permiten a un agente de IA trabajar con sistemas reales.

Prácticas · 2026

Ver proyecto +

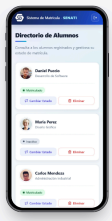


appEscuela

Directorio de alumnos en React para gestionar la matrícula.

Académico · 2026

Ver proyecto +

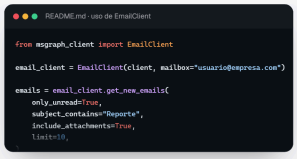


msgraph-Robin

Librería de Python para automatizar Microsoft 365.

Prácticas · 2026

Ver proyecto +



appTareas (doit)

Gestor de tareas para Android que funciona sin conexión.

Académico · 2026

Ver proyecto +

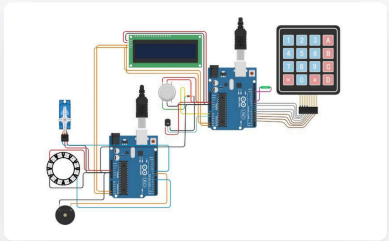


Alerta sísmica con Arduino

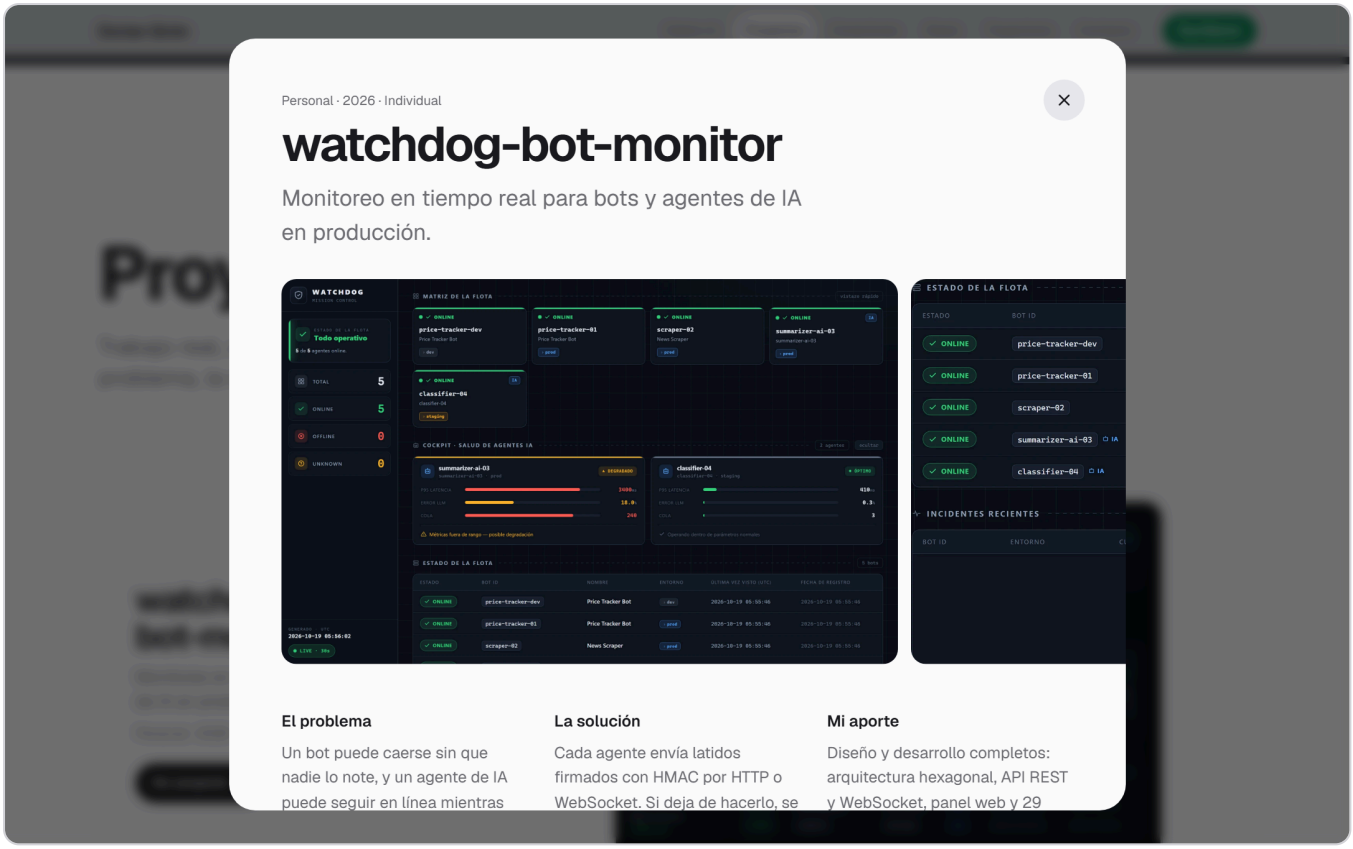
Sistema de alerta ante sismo, incendio y fuga de gas con dos Arduino conectados por I2C.

Académico · 2026

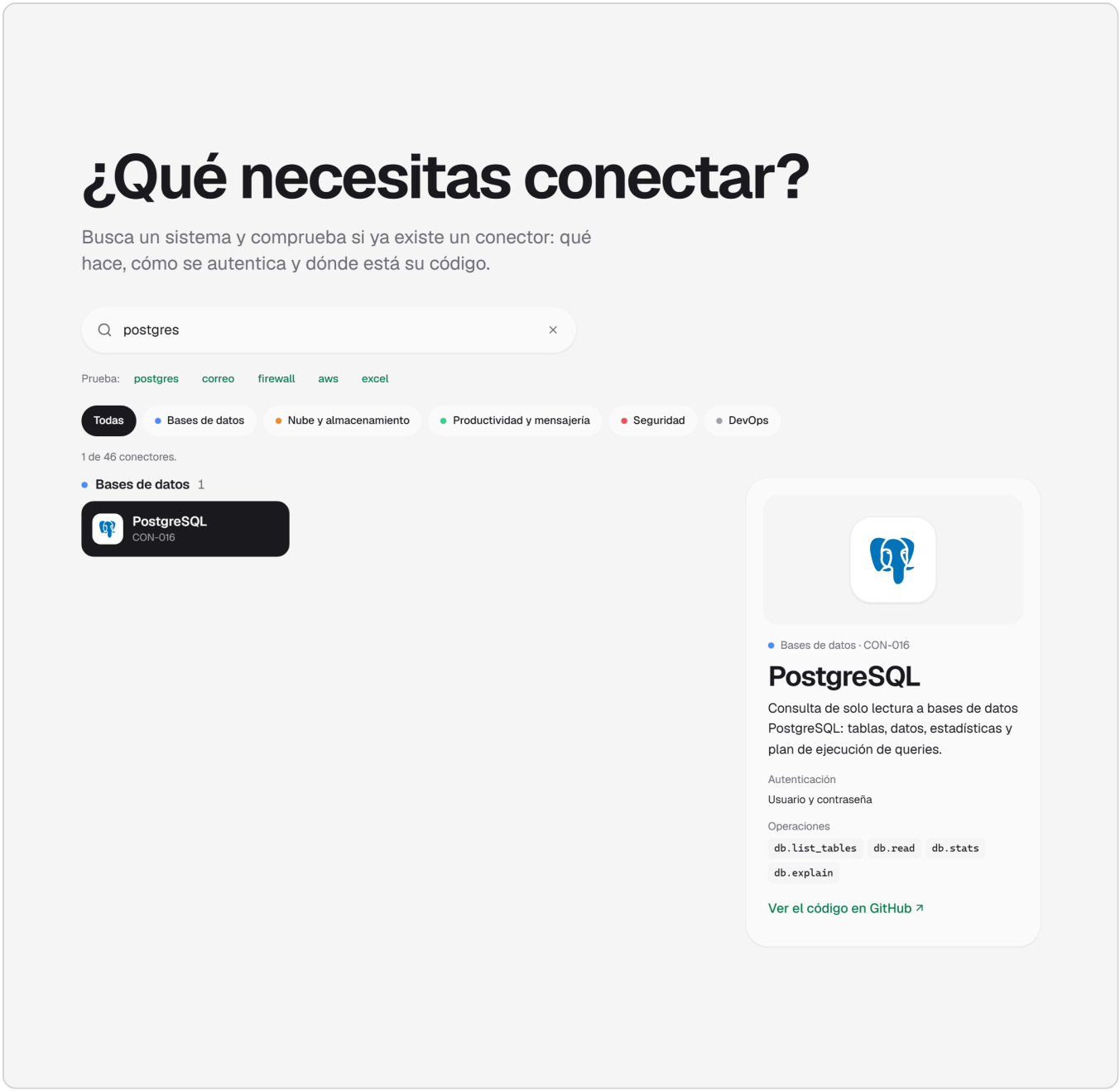
Ver proyecto +



Cuadrícula de proyectos.



Detalle de un proyecto en un diálogo modal.



Explorador de los 46 conectores con búsqueda en vivo (filtro «postgres»).

Pasos 17 a 19

Contacto: formulario controlado

Equivale a `Contacto.jsx`. Un único estado (`useState`) guarda nombre, correo y mensaje; `onChange` actualiza el campo por su atributo `name` (como `manejarCambio` en la guía) y `onSubmit` evita la recarga (como `manejarEnvio`).

> El sitio no tiene servidor, así que el formulario no finge un envío: abre la aplicación de correo del visitante con el mensaje ya redactado (`mailto:`) y lo indica en pantalla.

src/components/ContactForm.tsx

```tsx
import { useId, useState, type ChangeEvent, type FormEvent } from 'react'
import { Send } from 'lucide-react'
import styles from './ContactForm.module.css'

interface Fields {
  nombre: string
  email: string
  mensaje: string
}

const EMPTY: Fields = { nombre: '', email: '', mensaje: '' }

/**
 * Formulario controlado (un único estado con todos los campos).
 * No hay servidor: al enviar, abre la aplicación de correo del visitante con el mensaje ya redactado.
 * Así nunca aparenta un envío que no ocurre.
 */
export function ContactForm({ to }: { to: string }) {
  const [form, setForm] = useState<Fields>(EMPTY)
  const [status, setStatus] = useState<'idle' | 'opened'>('idle')
  const id = useId()

  const onChange = (e: ChangeEvent<HTMLInputElement | HTMLTextAreaElement>) => {
    const { name, value } = e.target
    setForm((prev) => ({ ...prev, [name]: value }))
  }

  const onSubmit = (e: FormEvent<HTMLFormElement>) => {
    e.preventDefault()
    const subject = `Contacto desde el portafolio — ${form.nombre}`
    const body = `${form.mensaje}\n\n${form.nombre}\n${form.email}`
    window.location.href = `mailto:${to}?
subject=${encodeURIComponent(subject)}&body=${encodeURIComponent(body)}`
    setStatus('opened')
  }

  return (
    <form className={styles.form} onSubmit={onSubmit} aria-describedby={`${id}-note`}>
      <div className={styles.row}>
        <div className={styles.field}>
          <label htmlFor={`${id}-nombre`}>Nombre</label>
          <input
            id={`${id}-nombre`}
            name="nombre"
            type="text"
            autoComplete="name"
            value={form.nombre}
            onChange={onChange}
            placeholder="Tu nombre"
```

```
            required
          />
        </div>
        <div className={styles.field}>
          <label htmlFor={`${id}-email`}>Correo</label>
          <input
            id={`${id}-email`}
            name="email"
            type="email"
            autoComplete="email"
            value={form.email}
            onChange={onChange}
            placeholder="tu@correo.com"
            required
          />
        </div>
      </div>
      <div className={styles.field}>
        <label htmlFor={`${id}-mensaje`}>Mensaje</label>
        <textarea
          id={`${id}-mensaje`}
          name="mensaje"
          rows={5}
          value={form.mensaje}
          onChange={onChange}
          placeholder="Cuéntame sobre tu proyecto o la oportunidad…"
          required
        />
      </div>
      <div className={styles.footer}>
        <p id={`${id}-note`} className={styles.note} aria-live="polite">
          {status === 'opened'
            ? 'Se abrió tu aplicación de correo con el mensaje listo. Solo falta pulsar Enviar.'
            : 'Al enviar se abrirá tu aplicación de correo con el mensaje ya redactado.'}
        </p>
        <button type="submit" className="btn">
          <Send size={18} aria-hidden="true" /> Preparar correo
        </button>
      </div>
    </form>
  )
}
```

src/sections/Contact.tsx · líneas 10–47

```
export function Contact() {
  return (
    <section id="contacto" className={`dark ${styles.section} defer`} aria-labelledby="contacto-title">
      <div className={styles.inner}>
        <Reveal>
          <h2 id="contacto-title" className={styles.title}>
            ¿Hablamos?
          </h2>
        </Reveal>
        <Reveal delay={0.1}>
          <p className={styles.lead}>
            Busco oportunidades como desarrollador y proyectos de integración o automatización. Si tienes sistemas que no
            se hablan entre sí, escríbeme.
          </p>
        </Reveal>
        <Reveal delay={0.2}>
          <div className={styles.actions}>
            <a className="btn" href={`mailto:${profile.email}?subject=${subject}`}>
```

```
              <Mail size={18} aria-hidden="true" /> {profile.email}
            </a>
            <ExternalLink className="btn btn-ghost" href={profile.links.linkedin.href}>
              LinkedIn <ArrowUpRight size={18} aria-hidden="true" />
            </ExternalLink>
            <ExternalLink className="btn btn-ghost" href={profile.links.github.href}>
              GitHub <ArrowUpRight size={18} aria-hidden="true" />
            </ExternalLink>
          </div>
        </Reveal>
        <Reveal delay={0.25}>
          <ContactForm to={profile.email} />
        </Reveal>
        <p className={styles.where}>
          {profile.location} · {profile.workMode}
        </p>
      </div>
    </section>
  )
}
```

Estilos (Paso 18)

src/components/ContactForm.module.css · líneas 18–59

```
.field {
  display: grid;
  gap: 8px;
}

.field label {
  font-size: var(--fs-small);
  color: var(--on-night-muted);
}

.field input,
.field textarea {
  width: 100%;
  padding: 14px 16px;
  border: 1px solid var(--night-3);
  border-radius: 14px;
  background: var(--night);
  color: var(--on-night);
  font-size: 1rem;
  caret-color: var(--accent);
  transition:
    border-color 200ms var(--ease),
    box-shadow 200ms var(--ease);
}

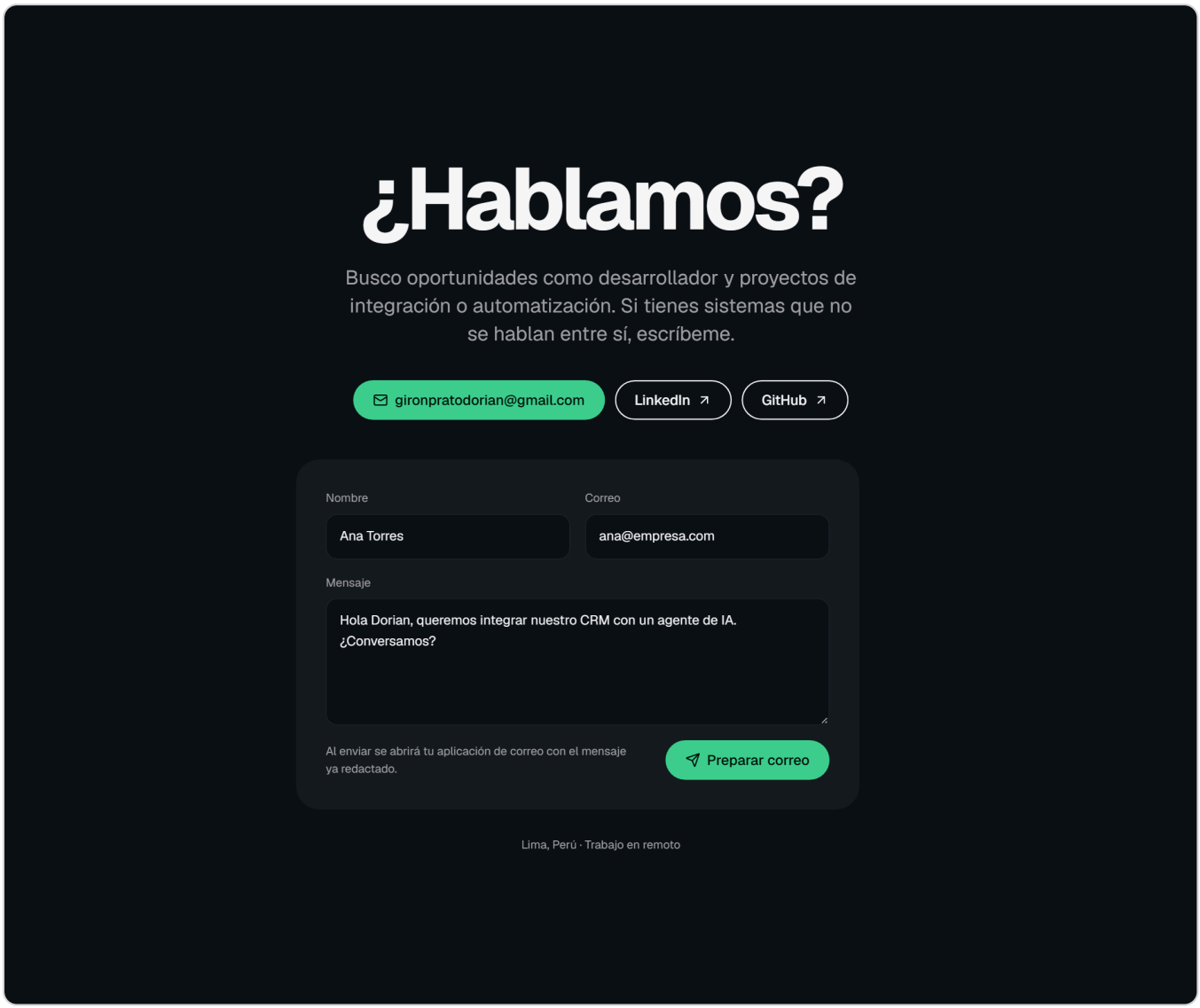
.field textarea {
  resize: vertical;
  min-height: 130px;
}

.field input::placeholder,
.field textarea::placeholder {
  color: #6e7681;
}

.field input:focus,
.field textarea:focus {
  outline: none;
  border-color: var(--accent);
```

```
57    box-shadow: 0 0 0 3px rgb(62 207 142 / 0.25);
58  }
```

Ejecución



Formulario de contacto con datos de ejemplo escritos.

Pasos 4, 7, 10, 13, 16 y 19

Ensamblaje en App

Como en la guía, cada componente se importa y se coloca en `App`, en el orden en que aparece en la página. `main.tsx` monta la aplicación.

src/App.tsx

```tsx
import { LazyMotion, MotionConfig, domAnimation } from 'motion/react'
import { SiteHeader } from './components/SiteHeader'
import { Hero } from './sections/Hero'
import { About } from './sections/About'
import { Statement } from './sections/Statement'
import { ConnectorWall } from './sections/ConnectorWall'
import { Projects } from './sections/Projects'
import { Explorer } from './sections/Explorer'
import { Stack } from './sections/Stack'
import { Journey } from './sections/Journey'
import { GitHubLive } from './sections/GitHubLive'
import { Contact } from './sections/Contact'
import { Footer } from './sections/Footer'

export default function App() {
  return (
    <LazyMotion features={domAnimation} strict>
      <MotionConfig reducedMotion="user">
      <a className="skip-link" href="#contenido">
        Saltar al contenido
      </a>
      <SiteHeader />
      <main id="contenido" tabIndex={-1}>
        <Hero />
        <About />
        <Statement />
        <ConnectorWall />
        <Projects />
        <Explorer />
        <Stack />
        <Journey />
        <GitHubLive />
        <Contact />
      </main>
      <Footer />
      </MotionConfig>
    </LazyMotion>
  )
}
```

src/main.tsx

```tsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './styles/global.css'
import App from './App'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

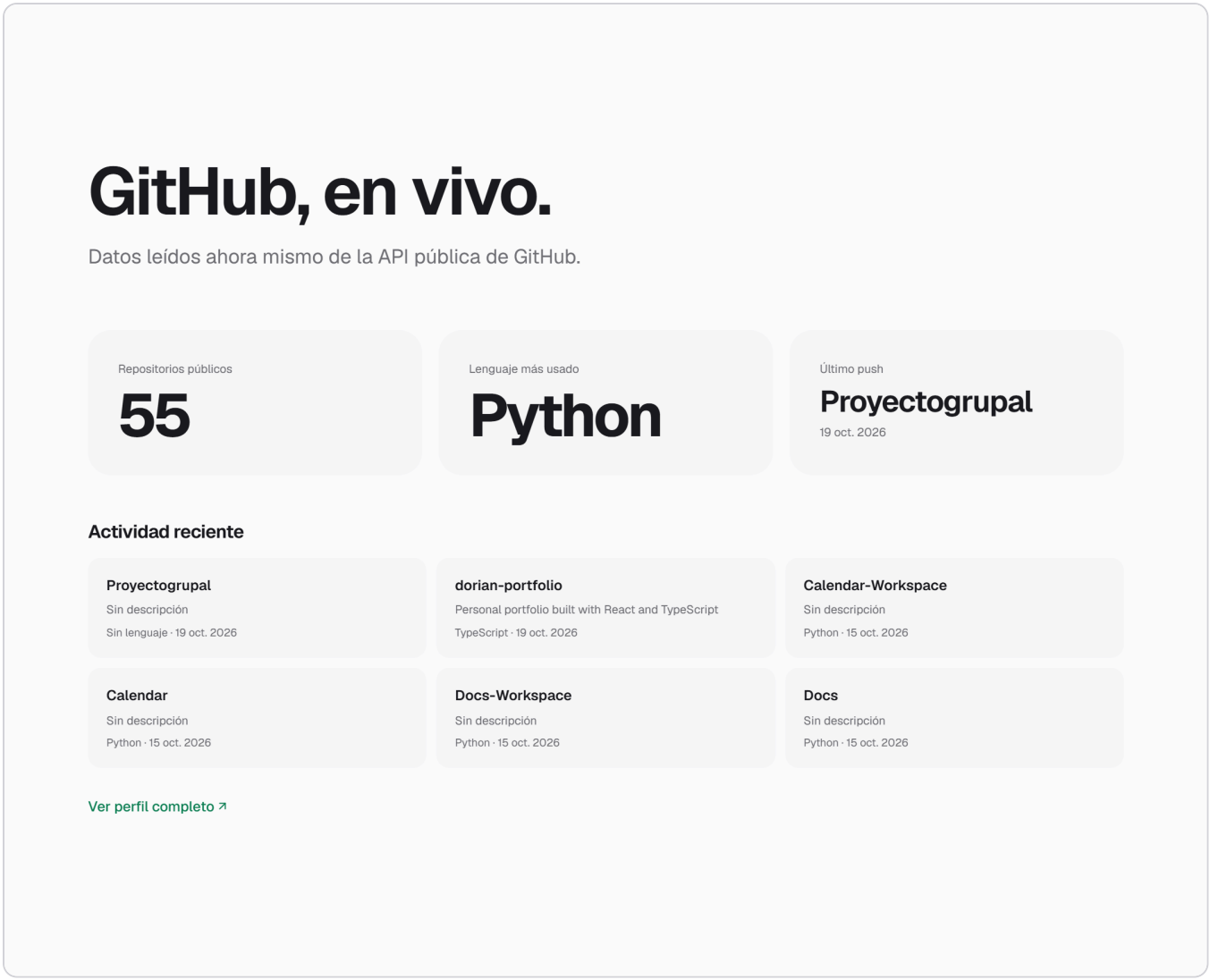
Vista móvil



Proyectos en móvil.



Contacto en móvil.



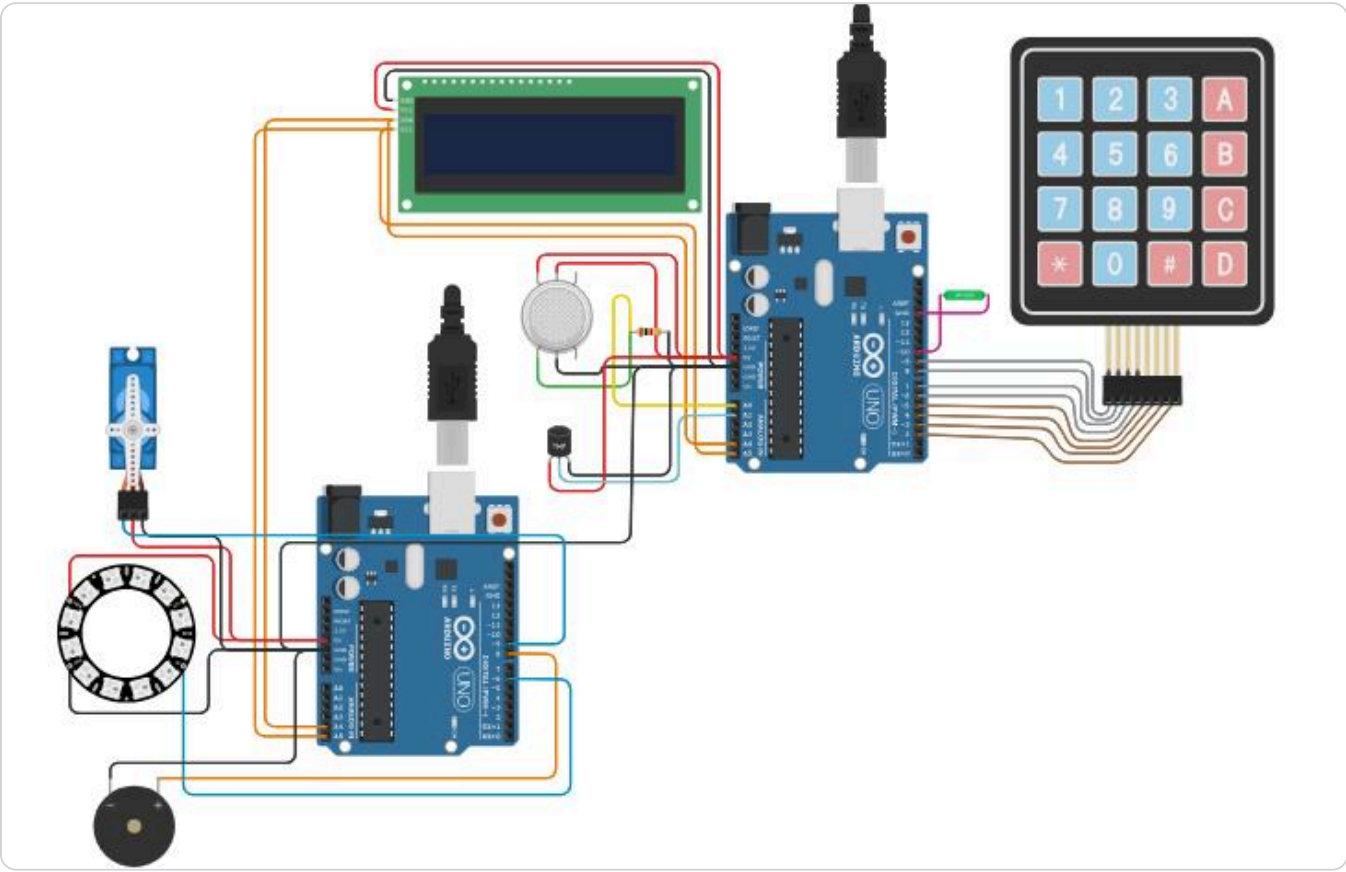
Actividad de GitHub en vivo, leída de la API pública.

Proyecto adicional

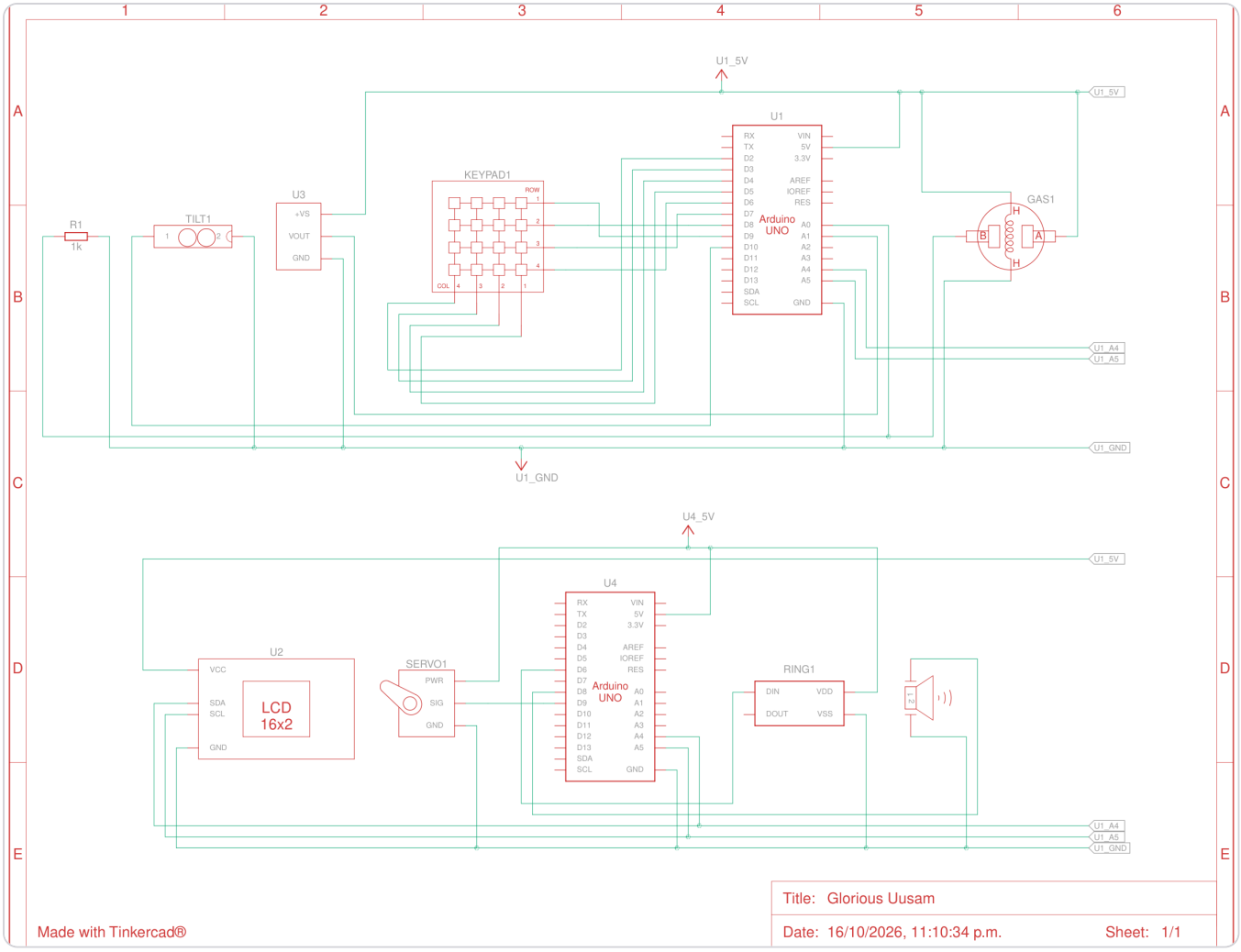
Alerta sísmica con Arduino (Tinkercad)

Circuito diseñado y simulado en Tinkercad Circuits e incorporado al portafolio como proyecto. Dos Arduino Uno se comunican por I2C:

- **Controlador (U1, maestro):** lee el sensor de inclinación, el de temperatura TMP36 y el de gas cada 500 ms, muestra los valores en un LCD 16×2 y envía el código de alerta (0 normal, 1 sismo, 2 incendio, 3 gas). La alarma se desactiva con una clave en un teclado 4×4.
- **Actuadores (U4, esclavo en la dirección 8):** abre la puerta de emergencia con un servo, anima un anillo NeoPixel con un color por evento y hace sonar el zumbador con un patrón distinto en cada caso.



Vista de protoboard en Tinkercad.



Esquema eléctrico exportado de Tinkercad. U1 y U4 comparten A4 (SDA), A5 (SCL), 5 V y GND.



El proyecto dentro del portafolio.

iot/alerta-sismica/controlador/controlador.ino

```ino
#include <Wire.h>
#include <Keypad.h>
#include <Adafruit_LiquidCrystal.h>

#define DIR_ESCLAVO 8

const int PIN_TILT = 10;
const int PIN_GAS  = A0;
const int PIN_TEMP = A1;

const int    UMBRAL_GAS  = 300;   // Ajustar mirando el Monitor serie
const float  UMBRAL_TEMP = 50.0;  // °C
const String CLAVE       = "1234";

Adafruit_LiquidCrystal lcd(0);    // LCD I2C (MCP23008, dirección 32)

const byte FILAS = 4, COLUMNAS = 4;
char teclas[FILAS][COLUMNAS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};
byte pinesFilas[FILAS]       = {9, 8, 7, 6};
byte pinesColumnas[COLUMNAS] = {5, 4, 3, 2};
Keypad teclado = Keypad(makeKeymap(teclas), pinesFilas, pinesColumnas, FILAS, COLUMNAS);

// 0 = normal, 1 = sismo, 2 = incendio, 3 = gas
byte alerta = 0;
int tiltReposo;
String entrada = "";
unsigned long ultimaLectura = 0;
float temperatura = 0;
int gas = 0;

void setup() {
  Wire.begin();
  Serial.begin(9600);
  pinMode(PIN_TILT, INPUT_PULLUP);
  tiltReposo = digitalRead(PIN_TILT);   // Estado sin sismo

  lcd.begin(16, 2);
  lcd.setBacklight(1);
  lcd.print("Alerta Sismica");
  lcd.setCursor(0, 1);
  lcd.print("Iniciando...");
  delay(1500);
  lcd.clear();
}

void loop() {
  char tecla = teclado.getKey();
  if (tecla) procesarTecla(tecla);

  if (millis() - ultimaLectura >= 500) {
    ultimaLectura = millis();
    leerSensores();
    if (alerta == 0) alerta = detectarEvento();
    enviarAlerta();                     // Se reenvía siempre, por seguridad
    actualizarLCD();
  }
}

```

```
void leerSensores() {
  float voltaje = analogRead(PIN_TEMP) * 5.0 / 1024.0;
  temperatura = (voltaje - 0.5) * 100;
  gas = analogRead(PIN_GAS);

  Serial.print("Temp: ");      Serial.print(temperatura);
  Serial.print(" C | Gas: ");  Serial.print(gas);
  Serial.print(" | Tilt: ");   Serial.print(digitalRead(PIN_TILT));
  Serial.print(" | Alerta: "); Serial.println(alerta);
}

byte detectarEvento() {
  if (digitalRead(PIN_TILT) != tiltReposo) return 1;  // Sismo (máxima prioridad)
  if (temperatura > UMBRAL_TEMP)           return 2;  // Incendio
  if (gas > UMBRAL_GAS)                    return 3;  // Fuga de gas
  return 0;
}

void enviarAlerta() {
  Wire.beginTransmission(DIR_ESCLAVO);
  Wire.write(alerta);
  Wire.endTransmission();
}

void procesarTecla(char tecla) {
  if (alerta == 0) return;               // El teclado solo sirve en alerta

  if (tecla == '*') {
    entrada = "";                        // Borrar
  } else if (tecla == '#') {             // Confirmar
    if (entrada == CLAVE) {
      alerta = 0;
      enviarAlerta();
    } else {
      lcd.clear();
      lcd.print("Clave incorrecta");
      delay(1000);
      lcd.clear();
    }
    entrada = "";
  } else if (entrada.length() < 4) {
    entrada += tecla;
  }
  actualizarLCD();
}

void actualizarLCD() {
  lcd.setCursor(0, 0);
  if (alerta == 0) {
    lcd.print("T:");   lcd.print(temperatura, 1);
    lcd.print("C G:"); lcd.print(gas);
    lcd.print("    ");
    lcd.setCursor(0, 1);
    lcd.print("Estado: NORMAL  ");
  } else {
    const char* nombres[] = {"", "SISMO!  ", "INCENDIO", "FUGA GAS"};
    lcd.print("ALERTA ");
    lcd.print(nombres[alerta]);
    lcd.print(" ");
    lcd.setCursor(0, 1);
    lcd.print("Clave: ");
    for (unsigned int i = 0; i < entrada.length(); i++) lcd.print('*');
    for (unsigned int i = entrada.length(); i < 9; i++) lcd.print(' ');
  }
}
```

iot/alerta-sismica/actuadores/actuadores.ino

```ino
#include <Wire.h>
#include <Servo.h>
#include <Adafruit_NeoPixel.h>

#define DIRECCION  8
#define PIN_NEO    6
#define NUM_LEDS   12
#define PIN_BUZZER 8
#define PIN_SERVO  9

Adafruit_NeoPixel luces(NUM_LEDS, PIN_NEO, NEO_GRB + NEO_KHZ800);
Servo puerta;

volatile byte alerta = 0;
byte alertaActual = 255;
int paso = 0;
unsigned long ultimoPaso = 0;

void setup() {
  Wire.begin(DIRECCION);
  Wire.onReceive(recibir);
  puerta.attach(PIN_SERVO);
  puerta.write(0);
  luces.begin();
  luces.show();
}

void loop() {
  byte nueva = alerta;
  if (nueva != alertaActual) {
    alertaActual = nueva;
    paso = 0;
    if (alertaActual == 0) {
      puerta.write(0);                // Cerrar puerta
      noTone(PIN_BUZZER);
      luces.clear();
      luces.show();
    } else {
      puerta.write(90);               // Abrir puerta de emergencia
    }
  }

  if (alertaActual != 0 && millis() - ultimoPaso >= 150) {
    ultimoPaso = millis();
    animarLuces();
    sonarSirena();
    paso = (paso + 1) % 240;
  }
}

void recibir(int n) {
  while (Wire.available()) alerta = Wire.read();
}

uint32_t colorAlerta() {
  if (alertaActual == 1) return luces.Color(255, 140, 0);  // Sismo: naranja
  if (alertaActual == 2) return luces.Color(255, 0, 0);    // Incendio: rojo
  return luces.Color(150, 0, 255);                         // Gas: morado
}

void animarLuces() {                  // Luces que "corren" hacia la salida
  luces.clear();
  for (int i = 0; i < 3; i++) {
```

```
    int pos = (paso - i + NUM_LEDS) % NUM_LEDS;
    luces.setPixelColor(pos, colorAlerta());
  }
  luces.show();
}

void sonarSirena() {                  // Un sonido distinto por evento
  if (alertaActual == 1) {
    tone(PIN_BUZZER, (paso % 2) ? 1200 : 800);
  } else if (alertaActual == 2) {
    tone(PIN_BUZZER, 600 + (paso % 8) * 100);
  } else {
    if (paso % 4 < 2) tone(PIN_BUZZER, 1000);
    else noTone(PIN_BUZZER);
  }
}
```

Calidad

Verificación

Salidas reales de los comandos del proyecto, ejecutados al generar este documento.

Terminal · dorian-portfolio

```
> npm run lint
(sin salida: sin errores)
```

Terminal · dorian-portfolio

```
> npm run typecheck
(sin salida: sin errores)
```

Terminal · dorian-portfolio

```
> npm test
RUN  v5.0.5 E:/proyecto portafolio


 Test Files  4 passed (4)
      Tests  15 passed (15)
   Start at  15:49:17
   Duration  254ms (import 42%, transform 23%, tests 20%, worker 14%, environment 1%)
```

Terminal · dorian-portfolio

```
> npm run build
✓ 2390 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                                              3.02 kB │ gzip:   1.17 kB
dist/assets/geist-cyrillic-ext-wght-normal-DjL33-gN.woff2    7.42 kB
dist/assets/geist-vietnamese-wght-normal-6IgcOCM7.woff2      8.00 kB
dist/assets/geist-cyrillic-wght-normal-BEAKL7Jp.woff2       15.08 kB
dist/assets/geist-latin-ext-wght-normal-DC-KSUi6.woff2      16.51 kB
dist/assets/geist-latin-wght-normal-BgDaEnEv.woff2          29.40 kB
dist/assets/index-BqR9K0NJ.css                              38.45 kB │ gzip:   9.29 kB
dist/assets/index-C7rMbVlu.js                              424.30 kB │ gzip: 136.15 kB
dist/assets/laptopScene-D0i9qtKk.js                        549.54 kB │ gzip: 137.56 kB

✓ built in 792ms
```

Lighthouse

Dispositivo	Rendimiento	Accesibilidad	Buenas prácticas	SEO	LCP
Escritorio	100	100	100	100	0.5 s
Móvil	92	100	100	100	2.4 s

Lighthouse 12.8.2 sobre la versión de producción (`npm run build` + `vite preview`), medido el 2026-10-19.

Control de versiones

Terminal · dorian-portfolio

```
> git log --oneline -10
601c717 Portátil fotorrealista y muro de conectores sin pausa al pasar el ratón
16a2d35 Portátil 3D realista con three.js, al estilo de Apple
5517343 Nueva foto en Sobre mí
961a6bc MacBook Pro e iPhone realistas basados en devices.css
d4ce25a Portátil 3D que se abre con el scroll e imágenes nítidas
0bd1ff2 Imágenes nuevas y explorador ordenado
8d99e84 Rediseño completo con experiencia de desplazamiento y proyectos en detalle
c78afa8 Portafolio personal en React y TypeScript
```

Cierre

Diferencias con la guía y conclusiones

Guía	Este proyecto	Motivo
JavaScript (`.jsx`)	TypeScript (`.tsx`)	Los tipos detectan errores en los datos antes de ejecutar.
Bootstrap y `.css` globales	CSS Modules	Estilos encapsulados por componente, sin choques de clases.
Datos dentro de cada componente	Datos en `src/data`	Para actualizar el portafolio solo se editan los datos.
Banner con foto circular	Portátil 3D que se abre con el scroll; la foto está en «Sobre mí»	Presentar los proyectos desde el primer vistazo.
Habilidades con porcentajes	Habilidades con evidencia	No inventar niveles: cada una enlaza a dónde se usa.
Formulario que limpia los campos al enviar	Formulario que abre el correo con el mensaje	Sin servidor, no se simula un envío que no ocurre.
Sección «Servicios»	Conectores, GitHub en vivo y Trayectoria	Mostrar trabajo real en lugar de servicios genéricos.

Conclusiones

- Dividir la página en componentes con su propio estado y estilos hace que cada parte se pueda modificar sin afectar a las demás.
- Separar los datos de la presentación permite añadir proyectos, como el de Tinkercad, sin cambiar la lógica de los componentes.
- `useState` resuelve tanto el menú móvil como el formulario controlado; `useEffect` se usa para los eventos del navegador.
- Las pruebas, el análisis estático y Lighthouse confirman que el sitio compila sin errores y es accesible.

> Código fuente: https://github.com/Dgirto/dorian-portfolio